



Javascript Generators and Iterators

Course Overview

Course Overview

Hi everyone. My name is Marques Woodson, and welcome to my course, JavaScript Generators and Iterators. I am a staff software engineer at project44 in Chicago. As JavaScript users, we use iteration almost every day. Understanding iterators and how they work is essential to being an efficient developer. In this course, we are going to learn all about JavaScript's built-in, as well as custom iterators, and generator functions. Some of the major topics that we will cover include how to create a custom iterator, old and new built-in iterators, how to pause and continue function execution with generator functions, and simplifying async logic with cancelable async flows. By the end of this course, you'll know what a built-in iterable is and why a for of loop works the way that it does. You'll understand how to create an iterator and why you'd want to use custom iterators in your code. Finally, you'll learn all about generator functions and how to pause and continue your function using an iterator. Before beginning the course, you should be familiar with basic JavaScript data structures and loops. Also, having some experience using the command line to run Node scripts would be helpful. I hope you'll join me on this journey to learn all about iterators with the JavaScript Generators and Iterators course, at Pluralsight.

Iterators and Iterables

What Are Iterators?

Hello. I'm Marcus Woodson, and this is my course JavaScript Generators and Iterators. We're going to be learning all about iterators, iterables, and generators. All of these features have been added to ECMAScript version 6 or just ES 2015 as it's more commonly called. As you'll see throughout this course, JavaScript now gives us new data structures and methods for natively dealing with their data, and iterating through data is not a new concept. For example, let's think about a loop. We use loops all the time to iterate over data. In a loop, we generally are iterating over each element within an array and doing something with each value. For loops and foreach loops are all you need for most situations. If you ever needed to stop iteration, that's possible by using a break keyword. This essentially stops the iteration, but there's no easy way to start it back up at a specific position. Sure, you can save the index and create a new



array with the rest of the elements you haven't iterated over yet, but that's a bit more complicated and adds a lot of extra logic. What if instead you just wanted to pause the iteration process and have the ability to unpause or resume the iteration whenever you wanted? This is what iterators can do. What are we going to learn in this module? In this module, we are going to talk about how an iterator differs from a standard loop in JavaScript. We'll also go over what an iterable is and what built-in iterables are available to us like arrays, strings, and new data structures like maps and sets. We'll talk about creating iterators from built-in iterables using the new primitive symbol. Finally, we'll go over creating custom iterators with custom iteration logic. So what defines an iterator? Well, technically, an iterator is any object that implements the iterator protocol by having a next method that returns a value property and a done property. The value property can be anything really, but the done property is a Boolean, which resolves to false when the iteration should stop. I could define a function here that creates a basic iterator that will run until a value is reached. If you haven't already, visit [CodeSandbox](#), [CodePen](#), or [JSPen](#) and follow along. Feel free to pause this clip until you're ready. I'll have two parameters, start and finish, which are both numbers. I'll define an index, which begins at the starting number and an iteration count starting at 0. Now, as I mentioned before, an object needs to have a next method that returns a value property and a done property. So I'm going to define and return a next method here, which will check to see if the iteration count is still less than the in number. And if it is, I'll increment the count and return the count as the value property while setting done to false since we still have not met the end yet. If the iteration count equals the end number, however, we set done to true. Let's create a variable to hold our iterator and start iteration by calling next right away. Now, I'll start a loop that continues to call next until the done property equals true. This done property works as an escape hatch in this loop. Without it, the loop would be infinite, and nobody needs that. Something that should be noted here is that iterators are lazy, meaning they are not going to do any work until you tell them to. This can be a huge performance benefit. In our example, we are just incrementing a number by one on each iteration. But imagine if you were doing something more complex, like compressing files or performing complex computations. Having the ability to pause at each iteration means you could break up the work in a more manageable way. Here you can see each value console logging until done is true. This is a basic example of creating a custom iterator. I don't think you should create iterators like this as there is no way to know that this object is an iterator. What you should do is create iterators from iterable objects. However, what is an iterable? Before I answer that question, I would like to introduce you to Console Log-My-Food. Console Log-My-Food is a CLI-based application that we're going to work on throughout this course. Let me paint you a scenario. Your manager comes up to you and says we are implementing a new health initiative here at the company. The head of HR has asked me if we had any engineers with enough bandwidth to help build an application. I told them I know the perfect individual. That's you. This application needs to do one thing, allow a user to log what they've eaten. HR plans to offer



this application as a free addition to the current benefits package for all employees in the hopes that we can help people get a better understanding of what they are consuming. So throughout this course, we will use what we learn in each module to build Console Log-My-Food. So let's get started creating this project in VS Code. Create a new folder called log-my-food somewhere on your machine. You can open this directory in VS Code by either typing code log-my-food or, if you've already moved into that directory, code period. Or if you don't have your VS Code set up with Bash, just open VS Code, then File, Open, and choose the log-my-food directory. Now that we are in the editor, let's get it ready to use npm packages. Open the terminal and run `npm init`. You can press Enter through these prompts. We really want to get a `package.json` in place so that we can install libraries later. Next, create a file named `index.js`, which will hold our application logic. We'll want to add a comment at the top of the file that lets the computer know that we want to execute this file with Node. So type out `#!/usr/bin/env node`. If you don't already have Node it installed, please do so using Homebrew if you're on a Mac, or just visit nodjs.org and download it there. Next, we need to make this file executable. So in the terminal again, type out `chmod +x ./index.js`. Now we can execute the file from the command line. If you are on a Windows machine, you won't have a `chmod` command to run as this works in UNIX shells only, like Bash for example. If you are running Windows 10, however, you can install a Bash cell. You can then run the same commands that I'm doing in this course on your Windows machine. If you don't have Windows 10, you will still need to install Node, but you can skip the `chmod` command. And every time you want to run the file, you'll add the `node` command before the file name. In the next clip, we'll talk more about some of the built-in iterables available and how we can create iterators using the `Symbol.iterator` method available on all iterables.

What Is an Iterable?

What exactly is an iterable in the JavaScript language? Well, without looking at any official definitions, I would imagine that it was an object that allows iteration of itself. Some type of array loop is what I would think of first since that's precisely what we are doing, iterating over the elements in an array. If you thought the same thing, then we are both partially correct. An array is a built-in iterable; however, they aren't the only iterables available, and there is more to being able to loop over an object that makes it an iterable. Specifically, for an object to be an iterable, it must implement the `iterator` method, meaning that the object must have a property with a `Symbol.iterator` key. Let me say that again. For an object to be an iterable, it must implement the `iterator` method, meaning that the object must have a property with a `Symbol.iterator` key. The `Symbol.iterator` is what JavaScript calls a well-known symbol, and its job is to specify the default iterator for an object. Now, this may sound a little new and abstract at first, but you've most likely used an iterator without even knowing it. Have you ever used a



for/of loop? If you answered yes, then you have experience with an object that has a `Symbol.iterator` key built in or a built-in iterable. The for/of loop only works with iterable objects. The for/of loop should not be confused with a for/in loop. The for/of loop iterates over the values of an iterable, like we'll see in this next example. A for/in loop, however, only iterates over an object's enumerable properties. For example, if you're using a for/in loop on an object, that would iterate over the keys of that object since the keys are the enumerable properties. Every time we call `it.next`, we are iterating one step at a time, printing the collection's next value to the console until we are done iterating. In this example, the fifth time we call `it.next` will let us know that there are no more values since the `done` property will be true. We can also do this with other built-in iterables. Let's try the same example using a map. Create a variable called `map`, set it to a new `Map`, then use `set` to add a few keys and values. Again, let's create an iterator, `mapIterator`, using the `Symbol.iterator` method available on the `map` structure, print out an iteration, and you will see the key-value pair as an array on each next call. When you set a key and a value in a map, they are stored as an array of key, value. If you were to use a for/of loop over a map, you'll see what I mean. You can see the connection between the `Symbol.iterator` method and the `next` call on the iterator that is produced. For/of loops are using the `Symbol.iterator` method under the hood and returning the value of the next call on each iteration. We can run this same example using a `Set` as well. Create a new set called `mySet`. I'll add two values to `mySet`, create an iterator with `Symbol.iterator`. Logging the `setIterator`'s next call shows us each value. This behavior is very close to the array-based example. Finally, a for/of iteration over `mySet` should work just like an array for/of. You may be wondering, why didn't `done` evaluate to true when I logged the last value of 6? Why did I have to call `it.next` again? Well, this is by design and considered a best practice. To see why this is the case, we need to look at how a for/of loop works. Here is the code to iterate over the iterator, `it`, that we created earlier. I'll bring in that iterator from earlier here as a reminder. We can use the for/of loop to log the values of the `it.next` output. We can write the equivalent functionality just using a standard for loop. After defining the value and result variables, we went to update result on every turn in the loop with `it.next` and only do this while `results.done` is false. You can see, this gives us the same output as the for/of loop, and you may see why when the result evaluates to the object with value 6, the `done` property can't also be true, and we need to call it one more time. If the `done` property were to be true, that value of 6 would basically be thrown out and never logged. This is why it's a best practice to call `it.next` one final time after going through each value. It's the default behavior of the for/of loop. If you were to pass a value when `done` is true in your own custom iterator, it could be confusing and throw someone off while looking at your code.

Iterating through JSON Data



I think it's time we start adding some functionality to our console.log my food app. We need to be able to run the app from the command line, have the app ask the user what food or drink they have eaten, and how much did they eat. Then, we'd like to show the user how many calories they've consumed. We have a few things here to set up, including the database of food items and the code to prompt the user in the terminal. Let's start with the prompt. Go ahead and open up the console.log my food project and open index.js. To deal with prompts, we're going to use the readline node module. From the node documentation, the readline module provides an interface for reading data from a Readable stream, such as process.stdin, one line at a time. We're going to instantiate a readline process where our input will be stdin and output will be stdout. This means we're taking input from the terminal and printing any output back to the terminal. Now let's use the readline question method, which takes a string as its first parameter and a callback. The parameter that has passed to the callback will be the input received from the user. Let's log out the message you have entered, then whatever input the user entered. We can end this program by calling readline.close. So for now, let's just have this simple program that we'll build off of. So how do we let the user know how many calories are in their food, and where are we saving their inputs? For this to work, we need some type of database. I recommend following along for this next part, so please feel free to pause while you are downloading and installing packages. We're going to use a tool called json-server. This gives us a full fake REST API. We're going to store nutritional information in JSON-formatted files and have access to them through REST requests. Let's get this installed using `npm install -g json-server`, or, if you prefer to not install this globally, you can instead use `npx json-server` once we are ready to start the server. To use json-server, we really only need a single file, that JSON file that holds our food and user info. I'm going to name the database file `db.json`. Go ahead and create that file with me. Let's create a `food` key, which is where we'll have a list of food items, and some basic nutritional information. We'll use an array of objects, and this first object will have an `id` of 1, a `name` of `apple`, I'll set the `calories` to 88 based on a small 6 oz apple. That size of a gala apple would be right around 88 calories. Copy and paste that same object two more times to save typing time. The second object will be for a chicken breast, so set the `id` to 2, the `name` to `chicken breast`, the `calories` to 125, serving size will be 3 oz. The last object will be whole milk, so increase the `id` to 3, set the `name` to `whole milk`, set the `calories` to 240, serving size will be 8 oz. I think this is a good starting point for our food database, at least for the purpose of this course. Next, we'll need a users database. We can add that to this file. Just create a new key below the `food` key called `users`. Again, this will be a list of objects, add a single user for now with an `id` of 1, a `first name` of `Sam`, and a `last name` of `Davies`. Also add an `email` key set to `sam@acme.com`. Let's go ahead and start the json-server by opening up the terminal and running the command `json-server -watch ./db.json -port 3001` from the root of the project. If you are using npx, you'll need to run `npx json-server`. Just put npx in front of the command. After a few seconds of waiting, your server should be up and running on port 3001. To test it out, you



can visit one of the resource links in a browser, and you should see the JSON data that we entered in `db.json`. Now, I want to show you how we can iterate through our food options to get the one item our user entered and display the calories for that food, all using what we've learned about the `Symbol.iterator` method. Back at our readline question of what would you like to log today, we can add some new logic here. We are going to want to make a GET request to our `json-server` food resource. So let's install `axios` with `npm i --save axios`. `Axios` is a promise-based http client that we'll be using to make RESTful requests to our `json-server`. Now, I like using `async await` when working with asynchronous calls. So first, let's add the `async` keyword to this readline callback. Then you can await an `axios` GET request to `localhost:3001/food`. Let's destructure the response to just get the data object, which will hold our array of food items. So remember, built-in iterables, like arrays, have the well-known symbol, `Symbol.iterator` method available, and calling this method on the iterable returns an iterator. That iterator will have the `next` method, allowing us to step through each item. To see that in action, create a variable to hold the iterator called `it` and create the iterator with `data` `Symbol.iterator`. This creates an iterator for us to use. To start iteration, we need to call `it.next`, and we'll save that value to a variable called `position`. Remember that the value returned by calling `it.next` will be an object with a `value` property and a `done` property. Let's start a while loop that runs until `done` is true. Inside the loop, we can get the current object of the iterable with `position.value`, and let's save the name from that object to a variable. We will check to see if the name of the food matches what the user answered, and if they do match, we'll print to the terminal and message telling the user how many calories that food has. We need to update the `position` variable with the next item in the iterator by calling `it.next` again. Once the iterator is finished and `position.done` is true, we close the readline and end the program. To recap, what makes an object an iterable? Well, it can be enumerated with the `for of` loop. It also must adhere to the iterable interface, which means there is a `Symbol.iterator` method on the object. The iterator returns a result that at a minimum must match an object with a `value` property and a `done` property. Other data can also be returned, but the built-in iterables will only return these two properties. Custom iterators have the ability to provide additional data that gets returned in a next iteration call. You can also have whatever logic you need to in your `Symbol.iterator` method. In the next clip, we will start building our own custom iterators, and instead of only iteration through simple data, we'll look at how we can iterate through a list of functions.

Custom Iterators

We've seen how we can use the built-in iterables to create iterators using the `Symbol.iterator` method. Each time we've used the symbol, it was just with its execution. What I want to show you now is how we can define that method with our own custom logic. This means that we will define both the `Symbol.iterator` and



the next function that it returns. But for what purpose will we use this custom iterator in our console log my food app. Well, I've got a new feature request that fits perfectly within custom iteration practices. We need to add the ability to list foods that are more specific to dietary preferences. What I mean by more specific is for a user who follows a vegan diet. They would like to list out foods that are considered vegan. Or if they wanted to see foods that fit into the ketogenic or keto diet, they can list those as well. Unfortunately, we aren't able to change our REST API to filter foods. So we need to do it ourselves. Before we add this functionality, we need to make our command line tool a little more robust. We need the user to have the ability to enter different commands for logging food and listing foods, to name a few. To turn this app into more of a CLI, we can use the readline prompt and line event listener. Head over to your editor, and the first thing we'll do is add a new option to the CreateInterface function. Enter prompt as the key and enter command > as the value. This means when we call prompt, the enter command string will print in the terminal. Go ahead and call readline prompt. Now we want to listen for user input. We do that with the line event listener. In the callback, add a switch statement, which will control the functionality based on the user's command. Add a log case, then copy and paste our readline question into the case statement block. Instead of calling close here, change that to readline prompt so that we return to the prompt after they log their food. Let's add a new case here called list vegan foods. For now, just console log vegan food list. We still need to add more foods to our database and create our custom iterator. Rather than typing all of the vegan foods in this video, I have included a JSON file with these items in the project files. For this course, you can grab those items and paste them into our db.json file before continuing. I've added a dietary_preferences array to each food in our list. We'll use this to know what category each item belongs with. Now let's get started with our custom iterator. In our list vegan foods code block, create a new object called veganIterable. Actually, we need to fetch the food from our API. So it makes more sense to define this iterator in the Axios promise's resolve. So first, make the same Axios get that we were calling earlier. And in the then method, let's create the veganIterable object. This object needs the Symbol.iterator method defined. And for it to be an iterable, it needs to return the iterator itself. The other method that is required is the next method. The next method needs to return an object with a value and a done property. We'll want to return each vegan food item one by one until there are no more items in the data array. But before we start filtering every non-vegan item out, let's just get the iterator working by returning all food items. In the next method, we want to get the current food item, so define a current variable to hold that. And we're going to need an index variable that we can increment. Let's put that at the top of this closure, idx = 0. Okay, so current will equal data[idx]. Then we'll increment idx on each next call. As long as current isn't undefined, we'll return an object where value equals current and done is false. If current is undefined, then there are no more items, and we return value as current still since it's undefined and done as true. That's all we really need. To make sure we have a working custom iterator, we can run it through a for/of



loop. I'll just let `val` of `veganIterable` here and console log `val.name` to just print the name of the food. When it's finished iterating, we'll call `readline` prompt again to get us back to the prompt. It's time to test this out. Head back over to your terminal and run `index.js`. Enter `list vegan foods` at the prompt, and hopefully you see a list of our food objects logged to the screen. It looks like we have successfully created a custom iterator, but we aren't finished yet. We really want to print out only the vegan foods. Right now, all foods are being returned. So let's go back to the code editor and update our iteration logic. What I'll do here is filter out all of the foods that don't include `vegan` as a dietary preference. Create a new variable called `veganOnly` and set that equal to the `data.filter` function. We'll make sure to only include foods where `vegan` is included in the dietary preferences array. Now just update the current value in the next method to use our new filtered array, `veganOnly`. Head back over to the terminal to test this out. Everything looks good. We are only seeing foods that would be considered `vegan`. I've actually added a few more items to the `db.json` file, like `almond`, `lentils`, and `oreos`. These items should be in the course files, so grab them from there. At this point in our project, we are using a custom iterator to iterate through normal data. But data isn't the only thing we can iterate through. What if you had a sequence of methods? Could we use an iterator for that? The answer is yes. Yes, we can.

Iterating Over Functions

Now that we know how to create a custom iterator, we should see what else we could do with it. Let's add to the log functionality of `console-log-my-food`. I'd like to have a follow-up question, if necessary. Sometimes people don't finish their entire serving. We want to adjust the calories when, for example, someone only eats half of a ribeye steak. This will give the user a better representation of what has actually been consumed. To make this work, we want to create a sequence of actions that will run when a user logs the food. We're going to do all of this with a custom iterator. Head over to the `index.js` file in your editor and go down to the log case. I'm going to do a little bit of a refactor here and move the `axios` call outside of the `readline.question` block. To continue using a `async await`, we'll need to add the `async` keyword up here at the line event listener callback. Now we can `await` promises in here. Right now, this log method will still work exactly as it did before. Let's add a new custom iterator here called `actionIterator`. We need to define the custom iteration logic within the `Symbol.iterator` method, so let's create that first. We know we want to make this iterator an iterable, so we'll have to return a `Symbol.iterator` method that will return itself. Now, how do we define a sequence of actions that will iterate through? To do that, we can create an array on this iterator to hold these actions. Let's call it `actions`. We'll populate this array with two functions, `askForServingSize` and `displayCalories`. In a moment, we'll define these functions. First, we need a place to store the current position in the sequence of actions. Define a variable called `positions`, and we'll just make a copy



of the actions array by spreading the contents into a new array. Now, what should happen within the next method? We know we need a next method. We also know that it needs to return an object that follows the iterator result interface. To iterate through each action, we will want to call one of our functions in the action array, depending on what position we are in. That means we should modify the positions array on each next call by removing that function, then calling it. That way, when the following next call happens, it won't accidentally call the same function twice. Once all of the functions have been called, we will set done to true. So let's start defining the next method now. First, check the length of the positions array to make sure we aren't finished. If there are still functions to call, let's grab the next function in the sequence by calling shift. Shift will take the first element of an array out of positions and return it. Next, we'll just call position to execute the function. Since some of the next calls can have arguments, we'll need to pass those in as args to the next method. So just spread the args in the next method, then also spread them at the position call. We also want to return done as false since we're not finished yet. If positions.length is 0, however, we'll want to return done as true since there are no more actions to execute. That's pretty much all we need right now with this next method. Now we need to define our two actions. We'll start with the askForServingSize method. In this function, we need to prompt the user with a question asking them, how many servings did they eat? We'll use readline.question here and ask the question, how many servings did you eat? And in parentheses, I'm going to ask the user to enter this value as either an integer like 1 or a decimal. This way, we can use simple math to calculate how many calories they are logging. In the callback, we are going to call next on the actionIt iterator. With the user input and the food object as the only two arguments, we'll get the food object when we first create the actionIt iterator and call next for the first time. We'll pass the food object to next at that time. The second action is called displayCalories. This function will take two arguments, the servingSize and the food object. Here we want to log to the screen the amount of calories associated with the chosen food. The sentence will read as food A with a serving size of X has Y calories. Since the servingSize can be a decimal, we'll need to make sure it gets parsed correctly as an integer. ServingSize is also a string, so we will also need to parse that as an integer. Now we can call actionIt.next again and return to the readline prompt. When calling actionIt.next, done will be set to true, as there are no more actions to perform. To start this iterator, we need to create the actionIt iterator when the user enters the food they are logging. Let's create a variable at the top of the log case called actionIt, which will hold the actionIterator instance. Now back in the What would you like to log today prompt, when we find the food that they entered, we want to create this iterator and call next for the first time. We'll assign the iterator to the actionIt variable that we just created. Call actionIt.next and pass in position.value, which is the food object that we need in our actions. At this point, everything should be set up for testing. So head over to the terminal and start up console-log-my-food. Type log, then enter a food like oreos, enter 2.5 as the serving size, and there's the message that we are expecting. Oreos, with the



serving size of 2.5, has 460 calories. Now you see how we can use a custom iterator to run through a sequence of functions. Next, we'll look at how we could've exited the action iterator early, using the optional return method that can be defined on a custom iterator.

Return Method and Saving Logs to a Database

So far, we've been going over creating a custom iterator and talking about what the object interface looks like. I thought it would be a good time to look at what the official spec defines as the iterator interface. This is what we've been implementing with our custom interface. The next method must return an object that follows the iterator result interface. There's also these optional methods, `return` and `throw`. Both of these return and iterate a result, and typically you would set the `done` property to `true`, but that's not a hard requirement. Something you should remember is that most built-in iterators will not have these optional methods available. Arrays, maps, strings, sets, none of these have a `return` method or a `throw` method that you can call. Later, we will talk about what type of iterator will have these methods available, but for now, we will be implementing these methods ourselves on the action iterator. In this clip, I'll demonstrate the `return` method and the `throw` method and how we can use them to exit the iteration process. We'll also start saving the food logging to the database so that later we can display what has been logged. To see the `return` method in action, let's give our program the ability to exit based on the user input. How about when the user is asked about how many servings they've eaten, we allow the user to enter the word `nevermind` if they would like to exit the question. Let's add some more logic to the `askForServingSize` function. We can check if the user has entered the word `nevermind`, or just the letter `n`, and if they did, we'll call `actionIt.return`. All other cases will still call the normal `actionIt.next`. For this to work, we will need to implement the `return` method on our action iterator. Define the `return` method right after the `next` method here. We want to make sure to empty the `positions` array, and then we can just return an object with the `done` property set to `true`. We are setting `positions` to an empty array just so that a rogue `next` call doesn't continue with the remaining functions since, remember, we do have a `positions.length` check in the `next` method. We also need to change this `const` into a `let`. We might as well implement the `throw` method too. Now, according to the spec, this method does not necessarily need to throw an error exception. It can if you want it to, but it's not required. If you do not throw an exception, you do need to return an iterator result object, preferably with `done` set to `true`. To find the `throw` method below the `return` method here, and we're going to let it take one argument. This argument will be the error message that we will log to the terminal. We'll `console.log` that message, then return an object with an undefined value and `done` set to `true`. Okay, so we have a custom iterator with the `return` and `throw` methods defined. We are logging food to the terminal with how many calories have been eaten, but we aren't



saving that information yet. All we're doing is logging it. Well, let's update this functionality to actually save these logs for the user so that later they can view what they've eaten. Head over to the `displayCalories` function. We're going to add an `axios.put` request here before we end the iterator. Since we're using JSON Server as our database, posts and puts will actually update the `db.json` file for us with the request body. First, we'll need to fetch the users log. We'll update that log with a new entry, then post that new log back to the user. Let's make this function an `async` function. Next, after the `console.log`, and an `axios.get` for the user with an ID of 1. We'll get the `data` property from the response and assign the log to a new variable called `usersLog`. Just in case the log is undefined, we'll set it to an empty array. Now we can create the request body, which will hold the rest of the user data along with the updated log. Spread the data object into a new object here, then add the log array and spread the current log first. Now for the log entry, we'll use an object with a key of the current date timestamp. The value will be an object with the name of the food, the serving size eaten, and the total calories. Now we can define the `axios.put` request to the same URL. Pass the put body object as the second param, and for the third parameter, we should make sure to add a header for `Content-Type` and set that to `application/json`. Okay, let's test this out by opening up the terminal again and logging an apple this time. We'll do two servings. Now, if we check the `db.json` file, we'll have a new entry into our food log for Sam Davies. Well, you made it through the first module of the course. Congratulations! We went over a lot of different topics that hopefully led to you having a better understanding of iterators and iterables. We defined what an iterator is, as well as what an iterable is, and what the difference is. We took a look at some of JavaScript's built-in iterables and worked through some examples. Then we built our own custom iterator into the `CONSOLE-LOG-MY-FOOD` application. And since `return` and `throw` aren't available on most built-in iterators, we implemented them from scratch onto our custom iterator. The type of iterator where `return` and `throw` are available is the topic of the next module, Generators. So again, great work making it through this module, and next up we'll learn all about generator functions.

Generator Functions

Intro to Generator Functions

At this point in the course, you should have a solid understanding of iterators in JavaScript. It's now time to learn about a new type of function that produces an iterator. For all of you React developers who use Redux Saga, you already have a general idea of what we'll be going over, and that is generator functions. In this module, we'll be going over generator functions and how they work. We'll talk about the special `yield` keyword available in a generator function, along with the definition of `yield delegation`. We'll cover how to abort a generator or complete a



generator early. Finally, we'll learn how to handle errors within a generator function. The definition of a generator function is a function that can be paused and resumed at a later time while having the ability to pass values to and from the function at each pause point. Generator functions can run to completion, but they don't have to. Up until now, in JavaScript, a function always ran until completion. Now we can write generator functions that never stop "running". I put that in quotes because it's not like a generator function is stuck in an infinite loop. It's just that maybe we don't need the rest of the function to execute. The syntax of a generator function isn't all that different from a normal function. We just need to add an asterisk somewhere in between the function keyword and the name of the function. It's not really important where you put the asterisk. It just needs to be before the name. Go ahead and open up a code sandbox so we can write our first generator function. Define a function called `timestampGenerator`. If you're using Prettier, it will most likely position the asterisk for you, leaving you even less to worry about. Let's just `console.log` a timestamp. When we call this function, you may be wondering why didn't anything happen? Nothing logged to the screen. Well, executing a generator function does not execute the containing code. This is an important point to remember with generator functions. Executing the generator function alone does not execute its containing code. It's like if you wanted to watch a TV that is currently turned off. Executing the generator function alone is just like you picking up the remote control. You haven't turned the TV on yet. You've just been given the controller. Executing the generator returns an iterator, or the controller. When you want to turn the TV on, `it.next`, or press power on the controller. Now you've started watching TV. You've started the execution. When you execute the function, it returns an iterator that we can control. If you remember from the previous module, to start the iterator, we need to call `next`. Well, that's exactly what we have to do in this case, too. Now we see a timestamp getting logged. To show that what the generator function returns really is an iterator, let's try a `for of` loop. I'd expect we see a single timestamp being logged. So `for (const val of it)` and just `console.log` the `val` here. And yeah, it logs a single timestamp. What would happen if I removed this asterisk and tried the `for of` loop? We get an error, `undefined is not iterable (cannot read property Symbol(Symbol.iterator))`. This is still a very basic example of a generator function. There's an important piece we are missing that really shows us the power of a generator. We want to be able to pause this function. As it is currently, the entirety of the function will execute on the next call. It's a pause execution. We need the `yield` keyword. Using the `yield` keyword tells the function to stop and wait. While it's paused, nothing else will execute until another `next` call happens. Go ahead and delete the `for of` loop. Let's add the `yield` keyword to our `timestampGenerator` function. Then, after the `yield`, we'll print `execution continued`. When we first call `it.next`, we get the timestamp being logged. Now the function is paused, waiting for us to resume its execution. When I call `it.next` again, the rest of the code and this function is executed and we see the `execution continued` message. There are a couple of other actions that the `yield` keyword can perform. It can send the value to the iterator, and it can receive a



value from the iterator. That means you can assign a yield expression to a variable inside of a generator function and have the iterator send the value to be assigned to that variable. Yield expressions can be used where pretty much any other normal JavaScript expression is used. Let's head back to the code sandbox and send some additional time to our generator function using yield. Instead of console logging the timestamp at the beginning here, let's assign that value to a variable called `ts`. Now create a variable called `additionalTime` and set the value to yield. Let's log the value of `additionalTime`. We'll want to add any `additionalTime` to the timestamp we generated, but only if it's defined. So write a check to make sure `additionalTime` is defined, then add it to `ts`. Now we need to pass a value to this yield. So in the second `it.next` call, we'll pass in one minute's worth of milliseconds, `1000 times 60`. We can also update these console logs to be more descriptive. So I add `additionalTime` to the `additionalTime` log, add updated `ts` to the `ts` log, and let's add a log for the original timestamp value for comparison. We can also send the value to the iterator using the yield expression. To show this, add a new yield under the original `ts` log. We'll yield the `ts` value back to the iterator. Now we can assign that first `it.next` call to a variable called `originalTimestamp`. Log that out to see its value. We also need to add another `it.next` call due to the additional yield we added. Let's step through this iteration. Comment out all of the `it.next` calls for now. We'll activate each line one by one so we can see what's happening in the generator with these yield expressions. Uncomment the first `it.next` call and the `console.log` of the `originalTimestamp`. We've started the iterator and should see the log of the original `ts` coming from the generator function. And since we are passing the `ts` variable in the first yield here, we also see our iterator result object being assigned to `originalTimestamp`. The function is now just waiting for another next call. What do you think will happen when we call next again? I'll uncomment the next call now and nothing, but why did nothing happen? Well, it's sitting and waiting now at the `additionalTime` assignment. Here, I'll add another `console.log` after that first yield expression for more clarity. Now you can see that we made it past the first yield and are now waiting at the second yield. Now uncomment the final next call, and we get the rest of the function's logs. I want to show an example of a continuously running generator function. What this means is that we are going to have a generator function with the while loop that does nothing but yield values. Create a new empty file called `randomNum.js`. If you created this sandbox from the vanilla JavaScript template, you'll need to update the `index.html` file script tag near the bottom to `.2 randomNum` instead of `index.js`. Define a generator function called `randomNum` at the top. And inside, we will add a while loop with an expression of `true` so that it will always run. Inside the loop, add a yield, and we're going to yield a random number between 0 and 100. So type out `Math.floor(Math.random() * 100)`. Since yield will always pause the execution of the function, this while loop won't end up crashing our browser. It will just give us a function that will always give us a new random number whenever I call it. Execute `randomNum` and save the iterator to `it`. Now every time we call `it.next`, we will get a new iterator result object with a random number. If



you wanted to abstract away the need to call next over and over, we could put this iterator into another function, which returns that value for us. Create a function named `getRandomNumber`, and inside, we will return the value of an `it.next` call. Now we just need to execute `getRandomNumber`. Up next, we'll see more examples of generator functions and introduce yield delegation.

Converting Iterators to Generators

Now that we know what a generator function is and how to implement one, we should update some of the `CONSOLE-LOG-MY-FOOD` code. I think we can reduce the amount of code we have written while at the same time, make it more readable. Let's take the list vegan foods case statement and use a generator in place of the custom iterator. The first thing we should do is use `await` here instead of the callback, assign the data variable from the `await`'s response here. Now define a function named `listVeganFoods`, and don't forget the asterisk to make it a generator. You can delete the custom iterator `veganIterable`, but leave the `for/of` loop for now. We'll keep the `idx` variable and the `veganOnly` variable too. Move those inside of our generator function. To make sure our `for/of` loop can still print out each vegan food, our generator function will need to pass a food out to the iterator using the `yield` keyword. In our previous custom iterator, we were returning that food item in the next method as long as it was defined in the `veganOnly` array. We're going to do something very similar here, but we'll use a while loop. While `veganOnly` still has items available, we'll yield out that item. After we yield, we need to increment our `idx` variable to get the next item. Now we just need to update the `for/of` loop to iterate over the iterator returned by the `listVeganFoods` generator. With these changes, we should see the exact same functionality that we had with the custom iterator. Head to the terminal to verify that everything still works. Yep, we're still looking good. The next custom iterator we defined was in the log case, the `actionIterator`. We used this iterator to cycle through an actions array. By instead defining this functionality in a generator function, it'll look more like a standard function call. Instead of an actions array, we would adjust the yield between the functions to have the same functionality. We also wouldn't have to define custom `return` and `throw` methods as those are both defined on generator functions already. Let's start creating a new generator function to replace `actionIterator`. Go ahead and delete the `actionIterator` object and define a generator function called `actionGenerator`. We'll still need to make sure the food the user enters gets passed into the generator, so we should set our first yield at the top and assign the value passed in from the iterator to the variable `food`. Okay, quick quiz. How many times would `next` need to be called before food gets assigned? The answer is twice, once to start the iterator and the second time for this first yield. It's important that you remember that the number of `next` calls will not equal number of yield statements. There will always be one extra `next` call to just start the iterator. Now, back to the `actionGenerator` function. The next line, we'll want to ask the user for



the serving size and save their entry, so add a yield expression here where the expression is the askForServingSize function. Since askForServingSize doesn't really use the food object, except for passing it to the displayCalories function, we don't need to pass it as a parameter. We can pass the food object to the displayCalories function during the yield expression. Update the actionIt.next call to only pass servingSize. The last step is displaying the calories, so just yield to displayCalories function with the servingSize and food object as parameters. This covers all of the actions we were calling in the custom iterator. To start this iterator, we need to update the actionIt variable by replacing the actionIterator call with the actionGenerator call in the readline.question. Now we call next to start the iterator. There is no need to pass the food object with the first next call since there is no yield expression waiting for a completion. The function has to run to its first yield, which is waiting for a food value to be passed back, so call next again on the actionIt and pass position.value back to the generator. We can run the program to make sure everything is still working as expected, and we're good. Coming up, we will add another feature to our program, the ability to list out what foods you logged for the current day, but first, let's talk about yield delegation.

Yield Delegation

In a generator function, the yield keyword is used to pause execution and to send values back out to the iterator. Since yield can send basically any value back to the iterator, I used to think, what if I wanted to send the values of a different generator function from this yield? That's when I learned about yield delegation. Yield delegation essentially allows a host generator function to control the iteration of a different generator function. I like to think of it as composing generator functions. Open up your code sandbox if you'd like to follow along here as we are going to write some examples of yield delegation. Let's define a generator function called gen1. And in this function, we'll yield a couple of values, 1 and 2. Now create a second generator function called gen2. Here is where we will use yield delegation. Type out yield* gen1. Yield* requires an iterable, which is provided by invoking gen1. Now let's get the iterator from gen2. Underneath that, place a console log it.next call. This starts up the gen2 generator and runs up to the first yield statement, which in our case is a yield delegator. That delegator has started up the gen1 iterator and run to its first yield statement, which is sending a value of 1. Calling it.next again should take us to the yield 2 statement in gen1. At this point, gen1 has run to completion. Let's add another yield statement below the yield* in gen2. Then call console log it.next again. Since gen1 has completed, gen2 has moved on to its own internal yield statements. What if you want to return a value from gen1 and save it to a variable. In gen1, you can add a return statement. Let's return 4. In gen2, we'll yield val. And if we console log it.next again, we can see the value 4 appear. Now we are passing a generator function to yield*, but the iterable that is required doesn't



have to be a generator function. You can pass any built-in or custom iterable. Let's remove the asterisk on `gen1` to make it a normal function. Then, instead of yielding something, just return an array of strings. Also remove the two extra `yields` from `gen2`. Now our console logs should reflect the strings returned in `gen1`. Now that we understand what yield delegation is, we should be able to implement it within our console log my food app. We've actually received a feature request, which requires the application to be able to list out the foods we've logged for the current day. Head over to your editor and add a new switch case called `today's log`. The way we want this to work is the user would type `today's log`. Then they'd enter their email address so we know what log to retrieve. We should then log out all of the foods eaten today, along with the total calorie count. First thing we'll need is a refined question. The text will just say email. The user would just type out their email and hit Enter. Now we'll need an Axios get request to the users database, and we'll filter by email address. Quick note. JSON Server gives you the ability to do this type of quick filtering right out of the box. We know each user object has an email field available, so passing email as a query parameter means we want to filter all users by the provided email address. We'll grab the data object off of the response, which will be an array of user objects. We will want the log from the first and only object on this array. Assign that to a variable called `foodLog`. Then create a variable called `totalCalories` with a value of 0. Now we want a generator function that will yield out all of the food logs from our user. So create a generator function named `getFoodLog`. And the only line we'll need inside is a yield delegator for `foodLog`. `FoodLog` is an array, meaning it's an iterable, which is required for yield delegation. We're going to use the `getFoodLog` iterator to iterate over the `foodLog` iterable. This is the yield delegation. We can use a `for/of` loop over the iterator returned by `getFoodLog`. We need to get the key of each entry, which is a timestamp from when the food was logged. So create a variable called `timestamp` and set that to `Object.keys` entry and get the first item from the array. We need to check if the timestamp's day matches the current day. Let's make a quick helper function called `isToday` that will take in a timestamp and return true or false if that timestamp is from today. Create a variable, `today`, and set that to a new Date object. Then we'll return three different equality checks. We'll check that the timestamp date matches today's date. We'll check if the timestamp's month matches today's month. And we'll check if the timestamp's year matches today's year. If all of those checks are true, then the function returns true. Otherwise, it'll return false. Back in our `for/of` loop, we'll check if the timestamp is today. The timestamp will actually be in string form, so we need to read it as a number first. Then create a new date object from the timestamp so that we can get the date, month, and year off of it. If the timestamp is today, we'll log out the message food eaten and the serving size. Then we'll increment the `totalCalories` variable. Outside of the loop, let's log a divider line, then log the total calories. Put a readline prompt at the bottom here to bring the user back to the prompt. Now let's head over to the terminal and create a few more log entries for today. Now we can type out `today's log`, enter



sam@acme.com since that's our only user and see our food log for today. And it looks like I have a typo somewhere. So let's go and fix that really quick. Now trying again, and there we see our day's food log. Using yield delegation to log out the food list gave us the benefit of not really having to care about how the food log was made. As long as the foodLog var is an iterable, our application would still work. We've abstracted the logic away from the generator that we are calling getFoodLog. Hopefully, you can see the benefits of yield delegation here and why it can be useful. Next up, we'll find out how we handle errors within a generator function, and how can we complete the generator function early using a return method.

Error Handling and Early Completion

Back when we were creating custom iterators, we talked about the optional methods return and throw. We implemented those manually before, but with generator functions, those are already included. We don't need to implement those ourselves. As a reminder, we use return or throw to abort the iterator before it would have normally completed. You'd use return when, for instance, you reached a pause point in the generator function, and you have a value that tells you that you don't need to continue to the rest of the pause points. We're done, return. A throw, however, would be used if you would like to trigger an error within a generator. This is useful when you are consuming the iterator provided by the generator in a separate context, like maybe you have some promise that throws an exception and you want to stop the execution of your generator from the promise's catch block. If the name of your iterator was its, you would just do its.throw from that promise's catch block. Don't worry if you don't fully understand the use of these methods yet, we'll be using these both in console-log-my-food. We actually already have a return in the code right now. If you remember, we added an escape key word while logging food at the servingSize step. If you enter the word nevermind or just the letter n for your servingSize, we call actionIt.return, which ends the generator sequence. Let's add another use of the return method. Take a look at our getFoodLog function. A new feature request has just come in. We are going to need to show a message to the user if their calorie amount goes over a certain value. For the sake of this example, we'll make that value 12,000. Those are world's-strongest-man eating numbers. If the user's calories go over 12,000, we'll print out a message to the console. First thing we'll want to do is create a variable to hold the iterator for getFoodLog. Let's call it logIterator. Now, in the if statement below the line where we increment the total calories, let's add another if block to check if total calories is above 12,000. If it is, log the message Impressive: You've reached 12,000 calories. Then call logIterator.return. With this return in place, the generator will not log any more food items to the screen. Let's go ahead and create a few high-calorie logs for the day. Log that you had chicken breast, 20 servings' worth. Then maybe let's have 15 servings of apples, then lentils, 5 servings. Add cheddar cheese at 10



servings Add whole milk with 10 servings. Add 10 servings of ribeye steak then add 8 servings of almonds. This should get us over the 12,000 calorie limit. Run the today's log command, and you'll see that after the ribeye was added, we went over 12,000. Since we returned and stopped logging foods, we won't see the almonds that we also logged. When we want to end a generator due to an error, we can use the throw method. Before we add this to console-log-my-food. I want to show you an example of using throw over in CodeSandbox. Create a generator function named errorHandling, add a try-catch block, and in the catch block we'll just log out the error message. In the try block, add three fields. Now let's create an iterator called errorIt by calling errorHandling. Log out the value of the first next call, which should be 1. Next, call errorIt.throw with an error message. As you can see in the console, our error message is reaching the catch block and logging. Try calling errorIt.next again, and you'll see that the number 2 is not logged, but instead done is true. Calling throw ends the generator execution. The same would happen if we were to throw an error inside the try block. Remove the errorIt.throw call, and instead place a normal throw call right after the first yield. Since we're already calling errorIt.next twice, you'll see right away that throwing inside the try block ends the generator in the same way. This is similar to how error handling works with async await functionality. Okay, let's look at the log functionality and handle the case where the user enters a serving size that is not a number and is not our escape hatch string of nevermind or the letter n. We'll want to call throw on the iterator, which will stop the generator from reaching the displayCalories action. First we'll want to add a try-catch block to the action generator to handle any errors thrown. Let's wrap what we currently have in the function in the try section. In the catch block we'll console.log whatever error message is passed into the throw command. Now we need to add an additional else if condition in the askForServingSize function. After checking for nevermind or n, add an else if. Here we'll check if the typeof servingSize is not a number or if the servingSize is NaN. If either of these are true, we'll call actionIt.throw with the error message, Please, numbers only. We really should update our other generator functions with try-catch blocks for better error handling. Go up to the listVeganFoods function, and let's wrap that logic in a try block. For the catch block, let's just print out the error message, Something went wrong while listing vegan items, along with the error object. The last generator function is the getFoodLogFunction in the today's log section. Wrap the yield delegation in the try block, and in the catch block log the error object and the message, Error reading the food log. Well, you've made it through the generators module. Throughout this module, we took what we learned about iterators and iterables and applied it to the implementation of generator functions. We learned that generator functions return iterators that can be advanced with next calls or within a for of loop. We learned that generators introduce the yield keyword, which provides a pause point within the function and gives the iterator control of the function's flow. We talked about yield*, or yield delegation, which lets one generator function control the flow of another generator function. We went over how to complete a



generator before all code has been executed using the return method on its iterator. And finally, we learned how to handle errors within a generator function and how we can use the iterator's throw method to trigger an error. Coming up, we'll look at a library called Cancelable Async Flows, or CAF. CAF is a library that makes generator functions look and work like an async function that has the ability to be canceled. Async functions are usually used more often than generator functions, so for those who just prefer the look of an async function, CAF can be very useful. In the next module we'll learn how to use CAF.

Real-world Examples and Cancelable Async Flows (CAF)

Cancelable Async Flows

Before closing out this course, I wanted to give a quick overview of a simple library for working with asynchronous requests and generator functions. Cancelable asynchronous flow is a tool created by Kyle Simpson in an attempt to make it easier to work with async actions and generator functions by making them look and feel more like normal async functions. Kyle Simpson is famous for his Javascript work such as the You Don't Know JS book series, and has helped thousands of engineers, including myself, become better at writing Javascript. I highly recommend you check out his books online. You'll learn a ton from his work. So why would you need something like CAF? Async functions are familiar and easy to use, and for most cases, the best option when working with async requests. The one downside is the inability to externally cancel an async function once it starts running. When I say externally cancel, think of how we control the flow of a generator function using the iterator it produces. We call `iterator.next` or `iterator.return` to control the function. That type of external control doesn't exist for async functions. CAF works by wrapping a generator function in a CAF function, which will return a promise when called. Whatever value you are returning in the generator will be the value the promise will resolve with, so you'll need to return something in the generator. These functions can be cancelled by using the CAF cancel token. A cancel token has a signal object available, which you must supply to the generator function as its first argument. That signal listens for an external abort call, which when called will cancel the generator. It's probably best to see some actual code examples to better understand what's happening. So I'm going to use CodeSandbox again to create some example. CAF functions. Since we use CAF for async functionality, we're going to need an API to make fetch requests against. For this, we can use JSON server again, even with CodeSandbox. I'm using a node container template, which basically gives you your own server running within a container, terminal access included. I've also already added



the code necessary to get JSON server up and running. I'm using the same `db.json` file that we used in `console.log my food` just for simplicity's sake. Now, CAF's purpose is to make a generator function look and work more like async functions. And what is the most common action taken inside an async function? Async requests, most likely to an API. So we're going to write a generator function that makes an API request to fetch users. We'll again use `axios` as our async request library. Let's just fetch the user's data and return the `axios` promise. Now let's create a variable to hold the fetch iterator. Calling `it.next` will return the `axios` promise as the value so you can do `it.next.value.then`, and log the response data. There we see the `users` array, which is just one user. If we call `it.next` again, we can complete the generator function. Now let's add in CAF so that we don't have to call the generator to get the iterator. At the top of the file, require the CAF object. If you're using `CodeSandbox` and are getting errors related to a missing CAF polyfill, try to use version 7 of CAF instead of version 8. Now wrap the fetch generator inside a CAF function, set this all to a variable called `fetch CAF`. Delete the iterator we were using before. Since we're using CAF now, the fetch generator function needs to accept a cancel token signal as its first argument. To create a token, you instantiate it with `new Caf.cancelToken`. Now let's call `fetch CAF`, pass in the token signal, add `.then` and `console.log` the response. We're getting the same output as when we were calling the `next` method on the iterator, but we're writing it all as a promise. We aren't using the cancel signal yet, though. I want to show you how to cancel the promise, and to do that, we'll use `CAF.delay` and `CAF,nottimeout`. `CAF.delay` is basically a promisified `set timeout` with the ability to be canceled, `CAF.timeout` gives you a timeout token that is set to abort after a specified period. This is a very common use case that we deal with where we may want to cancel a promise if it's just taking too long to resolve. Let's update our token previously created with `CAF.canceltoken` by using `CAF.timeout` instead. Set the first argument to 300 milliseconds, and the second argument will be the string, this is taking too long. The second argument is a message that will log when the time elapses. Now let's add a delay to the generator function to simulate a long-running request. Add `yield CAF.delay` with the signal as the first argument and 400 milliseconds as the second argument. Why are we using `CAF.delay` and not a normal `set timeout`? Well, if we use a normal `set timeout`, we would have to take care of cancelling that timeout manually. There are manual ways to keep track of the timeout's name for clearing later, but this adds more work for the developer. Here's a perfect example of what I mean that I've grabbed from the CAF documentation. Here we have a function called `delay` that returns a promisified `set timeout`. There's a five-second delay waiting at the bottom here, which will abort the token when elapsed. The problem is in the main call, we're only waiting 100 milliseconds until that completes. The five-second timeout won't automatically cancel and will continue running until the full five seconds passes. Again, sure, we could try to handle canceling the `set timeouts` manually, but with CAF, we don't need to do that when using the timeout token. Now back in the code, we need to update the `fetch CAF` all by not only



passing the signal, but the entire token. This is necessary so that CAF can cancel not just the fetch CAF call, but also the delay call at the same time. If you were to rework the timeout token, we want all places it's been passed to get cancelled as well, and in this case it's the CAF.delay where we're passing it. Let's add a catch block to the fetch CAF call to make sure we're handling the case of canceling the promise. In our example, we are seeing that our function is being cancelled. The axios call is never being made and our catch block is logging the, this is taking too long, message.

Summary and Course Wrap-up

We've learned that cancelable async flows lets you use generator functions in your async flows so that you can easily cancel requests if needed. This involved creating and using a cancel token that is passed to the generator and used to listen for an external abort call. We used the CAF.delay and CAF.timeout to wire up cancellation of a timeout and the generator function at the same time using a single token. One of the most common use cases is using the timeout token to cancel an async call that is taking too long. That's super helpful. And that wraps up this course. I hope that you've gained a more solid understanding of generator functions and iterators. These are both relatively new to the JavaScript language, but have a lot of potential for everyday use. I really enjoyed making this course, and I hope you enjoyed taking it. You can follow me on Twitter @mwq27, and you can see some of my blog posts at marqueswoodson.com. Follow me here on Pluralsight if you want to receive updates when I post new content. And finally, if you develop web applications and want to test their UI using a tool other than Selenium, check out my course End-to-end Web Testing with TestCafe: Getting Started. TestCafe is a great tool for end-to-end testing that doesn't require WebDriver and supports TypeScript right out of the box along with the newest JavaScript features. You can test your application across all major browsers and mobile devices without the need for plugins or addons. Thanks again for watching. Happy Coding!